Name: Naveen Kumar P **Reg. No.:** 192412326

Experiment No.: 4

Experiment Question

Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Aim

To build a multilayer feed-forward Artificial Neural Network in Python, train it with the Backpropagation algorithm on the XOR dataset, and test its predictions.

Concept

An **Artificial Neural Network (ANN)** is a network of connected neurons arranged in layers: an input layer, one or more hidden layers and an output layer. Each neuron computes a weighted sum of its inputs plus a bias and passes it through an activation function (here, the sigmoid).

Backpropagation is the algorithm used to train the network. It works in two phases:

- **Forward pass:** the inputs are propagated through the hidden layer to the output layer to obtain the predicted output.
- **Backward pass:** the error between the predicted and the actual output is propagated backwards, the gradient of the error with respect to each weight is calculated using the chain rule, and the weights are adjusted by gradient descent to reduce the error.

This is repeated for many epochs until the error becomes small. A single-layer network cannot learn XOR because XOR is not linearly separable, but a network with one hidden layer can.

Dataset Description

The XOR (exclusive OR) dataset is used. It has 4 training examples, 2 binary inputs and 1 binary output. The output is 1 only when exactly one of the inputs is 1.

X1	X2	Output (y)
0	0	0
0	1	1
1	0	1
1	1	0

Network architecture: 2 input neurons → 4 hidden neurons (sigmoid) → 1 output neuron (sigmoid). Learning rate = 0.5, epochs = 10000.

Algorithm

Backpropagation Algorithm
1. Initialize all weights with small random values and all biases with 0.
2. Repeat for each epoch until the error is small:
   Forward pass:
    a. Hidden layer output: h = sigmoid(X.W1 + b1)
    b. Output layer output: o = sigmoid(h.W2 + b2)
   Error:
    c. error = y - o
   Backward pass:
    d. Output layer delta: d_o = error * o * (1 - o)
    e. Hidden layer delta: d_h = (d_o . W2^T) * h * (1 - h)
   Update:
    f. W2 = W2 + lr * (h^T . d_o)   b2 = b2 + lr * sum(d_o)
    g. W1 = W1 + lr * (X^T . d_h)   b1 = b1 + lr * sum(d_h)
3. Test the trained network on the input data.

Step-by-step Procedure

1. Import NumPy and define the XOR input matrix X and target vector y.
2. Define the sigmoid activation function and its derivative.
3. Set the network size (2-4-1), the learning rate and the number of epochs. Initialize the weights W1, W2 randomly (fixed seed) and the biases b1, b2 with zeros.
4. For each epoch, do the forward pass to compute the hidden and output activations.
5. Compute the error and the mean squared error (loss).
6. Do the backward pass: compute the delta of the output layer, then the delta of the hidden layer.
7. Update the weights and biases using the learning rate.
8. Print the loss at regular intervals.
9. After training, run a forward pass on all inputs, threshold the output at 0.5 to get the predicted class, and calculate the accuracy.

Formulae

Sigmoid activation and its derivative:

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \qquad \sigma'(x) = \sigma(x)\,\bigl(1 - \sigma(x)\bigr)$$

Forward propagation:

$$h = \sigma(X W_1 + b_1), \qquad o = \sigma(h W_2 + b_2)$$

Mean squared error:

$$E = \frac{1}{N} \sum (y - o)^2$$

Deltas (error terms):

$$\delta_o = (y - o)\, o\,(1 - o), \qquad \delta_h = (\delta_o W_2^{T})\, h\,(1 - h)$$

Weight update rule, with learning rate η:

$$W_2 \leftarrow W_2 + \eta\, h^{T} \delta_o, \qquad W_1 \leftarrow W_1 + \eta\, X^{T} \delta_h$$

Python Program

```
import numpy as np

# Training data: XOR problem
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])
y = np.array([[0], [1], [1], [0]])

# Sigmoid activation and its derivative
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(a):          # a = sigmoid(x)
    return a * (1 - a)

# Network architecture: 2 inputs -> 4 hidden neurons -> 1 output
np.random.seed(42)
input_size, hidden_size, output_size = 2, 4, 1
W1 = np.random.uniform(-1, 1, (input_size, hidden_size))
b1 = np.zeros((1, hidden_size))
W2 = np.random.uniform(-1, 1, (hidden_size, output_size))
b2 = np.zeros((1, output_size))

lr = 0.5
epochs = 10000

for epoch in range(1, epochs + 1):
    # Forward propagation
    hidden = sigmoid(np.dot(X, W1) + b1)
    output = sigmoid(np.dot(hidden, W2) + b2)

    # Error
    error = y - output
    loss = np.mean(error ** 2)

    # Backward propagation
    d_output = error * sigmoid_derivative(output)
    d_hidden = np.dot(d_output, W2.T) * sigmoid_derivative(hidden)

    # Weight and bias updates
    W2 += lr * np.dot(hidden.T, d_output)
    b2 += lr * np.sum(d_output, axis=0, keepdims=True)
    W1 += lr * np.dot(X.T, d_hidden)
    b1 += lr * np.sum(d_hidden, axis=0, keepdims=True)

    if epoch % 2000 == 0 or epoch == 1:
        print(f"Epoch {epoch:5d}  Loss: {loss:.6f}")
```

```
# Testing the trained network
print("\nPredictions after training:")
hidden = sigmoid(np.dot(X, W1) + b1)
output = sigmoid(np.dot(hidden, W2) + b2)
for inp, out, target in zip(X, output, y):
    print(f"Input: {inp}  Output: {out[0]:.4f}  Predicted: {int(out[0] >= 0.5)}  Actual: {target[0]}")

accuracy = np.mean((output >= 0.5).astype(int) == y) * 100
print(f"\nAccuracy: {accuracy:.1f}%")
```

Output

```
Epoch     1  Loss: 0.262916
Epoch  2000  Loss: 0.001788
Epoch  4000  Loss: 0.000705
Epoch  6000  Loss: 0.000431
Epoch  8000  Loss: 0.000309
Epoch 10000  Loss: 0.000239

Predictions after training:
Input: [0 0]  Output: 0.0095  Predicted: 0  Actual: 0
Input: [0 1]  Output: 0.9839  Predicted: 1  Actual: 1
Input: [1 0]  Output: 0.9849  Predicted: 1  Actual: 1
Input: [1 1]  Output: 0.0195  Predicted: 0  Actual: 0

Accuracy: 100.0%
```

Result / Conclusion

A 2-4-1 feed-forward neural network was built and trained with the Backpropagation algorithm on the XOR dataset. The mean squared error fell from 0.2629 in the first epoch to 0.0002 after 10000 epochs, and the network classified all 4 inputs correctly (accuracy 100%). Hence the Backpropagation algorithm was successfully implemented and tested.